

UE25CS251A
DIGITAL LOGIC AND COMPUTER DESIGN

TEAM DLCD

Department of CSE and CSE (AI & ML)

DIGITAL LOGIC AND COMPUTER DESIGN

ARCHITECTURE-Assembly Language

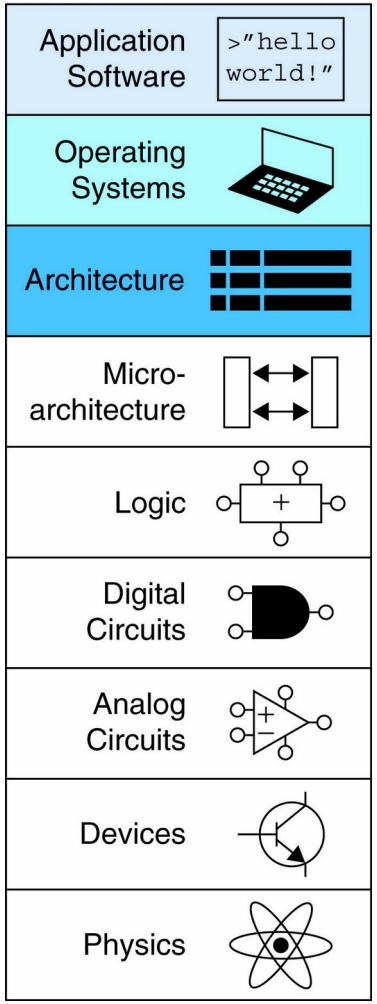
TEAM DLCD

Department of CSE and CSE (AI & ML)

DIGITAL LOGIC & COMPUTER DESIGN

TOPICS

- **Instructions**
- **Operands: Registers, Memory, and Constants**



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Assembly language

- **Instructions:** commands in a computer's language
  - **Assembly language:** human–readable format of instructions
  - **Machine language:** computer–readable format (1's and 0's)

- **MIPS** architecture:
  - **MIPS** (***Microprocessor without Interlocked Pipelined Stages***) is a family of reduced instruction set computer (RISC), instruction set architectures (ISA) developed by MIPS Computer Systems, now MIPS Technologies, based in United States.
  - Developed by John Hennessy and his colleagues at Stanford, in 1980's.
  - Used in many commercial systems, including Silicon Graphics, Nintendo and Cisco
  - Once you've learnt one architecture, it's easy to learn others

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MIPS Instruction: Addition

C Code	MIPS assembly code
a = b + c;	add a, b, c

add: mnemonic indicates operation to perform (here it is addition).

b, c: source operands (on which the operation is performed).

a: destination operand (to which the result is written).

Code Example 6.4 REGISTER OPERANDS

High-Level Code	MIPS Assembly Code
a = b + c;	# $s0 = a, $s1 = b, $s2 = c add $s0, $s1, $s2 # a = b + c

© Elsevier, Digital Design

DIGITAL LOGIC & COMPUTER DESIGN

MIPS Instruction: Subtraction

C Code	MIPS assembly code
a = b - c;	sub a, b, c

sub: mnemonic indicates operation to perform (here it is subtraction).

b, c: source operands (on which the operation is performed).

a: destination operand (to which the result is written).

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Architecture Design Principles

Computer architectures are designed using a set of fundamental principles that improve performance, simplicity, and efficiency.

1. **Simplicity Favors Regularity**
   - Simple and uniform instruction formats simplify hardware design.
   - Regular structures are easier to implement, test, and optimize.
2. **Make the Common Case Fast**
   - Frequently executed operations should be optimized.
   - Improves overall system performance since common tasks dominate execution time.
3. **Smaller is Faster**
   - Smaller hardware structures, such as registers and caches, have lower access time.
   - Faster access leads to improved processor performance.
4. **Good Design Demands Good Compromises**
   - Every design involves trade-offs between performance, cost, power, and complexity.
   - An efficient architecture balances these factors instead of optimizing only one.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Design Principle – 1

Simplicity Favors Regularity

- MIPS uses a consistent and uniform instruction format for most instructions.
- Arithmetic instructions follow the format:
  - Two source operands
  - One destination operand
- Uniform instruction formats simplify instruction encoding, decoding, and hardware implementation.
- Regular designs reduce hardware complexity, making processors faster, more reliable, and easier to design.

Example: add $s0, $s1, $s2

sub $t0, $t1, $t2

Both instructions follow the same format:

Destination = Source1 OP Source2

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Design Principle – 2

Make the Common Case Fast

- Processor performance is improved by optimizing the operations that are executed most frequently.
- MIPS includes a small set of simple, commonly used instructions that can be decoded and executed quickly.
- Simple instructions enable smaller, faster, and more efficient hardware.
- Complex operations are implemented by combining multiple simple instructions, rather than using a single complex instruction.
- MIPS follows the Reduced Instruction Set Computer (RISC) philosophy, emphasizing simplicity and speed.
- In contrast, architectures such as Intel x86 follow the Complex Instruction Set Computer (CISC) approach, where individual instructions perform more complex tasks.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Design Principle – 2

Feature	RISC (e.g., MIPS, ARM, RISC-V)	CISC (e.g., x86, VAX)
Instruction Set Size	Small, simple set (≈ 30–100 instructions)	Large, complex set (hundreds to thousands)
Instruction Length	Fixed (commonly 32 bits)	Variable (1–15 bytes in x86)
Execution Time	Most instructions complete in 1 clock cycle	Instructions may take multiple cycles
Complexity Location	Complexity shifted to compiler/software	Complexity handled by hardware/microcode
Memory Access	Load/Store architecture (only load/store access memory; ALU ops use registers)	ALU operations can directly access memory (read-modify-write)
Registers	Many general-purpose registers (e.g., 32 in ARM/RISC-V)	Fewer registers (e.g., 16 in x86-64)
Pipelining	Easy and efficient due to uniform instruction size	Harder due to variable-length instructions
Code Density	Larger program size (more instructions for complex tasks)	Smaller program size (complex tasks in fewer instructions)
Instruction Decoding	Simple, fast decoding	Complex, slower decoding
Hardware Complexity	Low — simpler control logic, fewer transistors	High — microcode, wide decoders, more transistors
Performance Strategy	Reduce cycles per instruction (CPI)	Reduce instructions per program
Power Consumption	Lower (efficient per watt, common in mobile/embedded systems)	Higher (desktop/server CPUs with more power budget)
Examples	ARM, RISC-V, MIPS, SPARC	Intel x86, AMD64, VAX

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Multiple Instructions

- More complex code is handled by multiple MIPS instructions.

C Code

```
a = b + c - d;
```

MIPS assembly code

```
add t, b, c      # t = b + c
sub a, t, d      # a = t - d
```

Code Example 6.5 TEMPORARY REGISTERS

High-Level Code	MIPS Assembly Code

`a = b + c – d;`	`# $s0 = a, $s1 = b, $s2 = c, $s3 = d`
	`sub $t0, $s2, $s3  # t = c – d`
	`add $s0, $s1, $t0  # a = b + t`

A RISC architecture minimizes the hardware complexity and the necessary instruction encoding by keeping the set of distinct instructions small.

For example, an instruction set with 64 simple instructions would need $\log_2 64 = 6$ bits to encode the operation.

An instruction set with 256 complex instructions would need $\log_2 256 = 8$ bits of encoding per instruction.

In a CISC machine, even though the complex instructions may be used only rarely, they add overhead to all instructions, even the simple ones.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Operands: Locations

Every instruction performs an operation on one or more operands. An operand is the data used by an instruction, and it can be stored in different locations within a computer.

The three operand locations in the MIPS architecture are:

- **Registers** – High-speed storage locations inside the CPU used for frequently accessed data.
- **Memory** – Stores large amounts of program instructions and data that cannot fit in registers.
- **Constants** (Immediates) – Fixed values encoded directly within the instruction.

Example: addi $t0, $t1, 10.

$t0 → Destination Register

$t1 → Source Register

10 → Immediate (Constant)

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Operands: Registers

- Registers are small, high-speed storage locations inside the CPU used to hold operands and intermediate results.
- MIPS provides **32 general-purpose** registers, **each 32 bits** wide.
- Registers can be accessed much faster than main memory, reducing instruction execution time.
- MIPS is called a **32-bit architecture** because its registers and ALU operate on 32-bit data.
- Frequently used variables are stored in registers, while less frequently used data is stored in memory.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Design Principle – 3

Smaller is Faster

- Smaller hardware structures generally have lower access time and can operate at higher speeds.
- MIPS includes 32 general-purpose registers, providing a balance between performance and hardware complexity.
- A smaller register file allows faster access, reducing instruction execution time.
- Frequently accessed data is stored in registers, while larger amounts of data are kept in main memory.
- This design improves overall processor performance by minimizing slower memory accesses.

Example:

- Accessing data from a register takes significantly less time than accessing the same data from main memory.
- Therefore, compilers try to keep frequently used variables in registers whenever possible.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MIPS Register Set

Name	Register Number	Usage
$0	0	the constant value 0
$at	1	assembler temporary
$v0–$v1	2–3	function return values
$a0–$a3	4–7	function arguments
$t0–$t7	8–15	temporaries
$s0–$s7	16–23	saved variables
$t8–$t9	24–25	more temporaries
$k0–$k1	26–27	OS temporaries
$gp	28	global pointer
$sp	29	stack pointer
$fp	30	frame pointer
$ra	31	function return address

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Operands: Registers

MIPS Register Naming Convention

- In MIPS, every register name is prefixed with a '$' symbol.
- Example: $0 (read as register zero or dollar zero).
- Registers are assigned specific roles to simplify programming and improve code organization.

Commonly Used Registers

- $0 (Register Zero): Always stores the constant value 0.
- $s0 – $s7 (Saved Registers): Used to store variables whose values must be preserved across function calls.
- $t0 – $t9 (Temporary Registers): Used to store intermediate results during calculations; their contents need not be preserved.
- $a0 – $a3 (Argument Registers): Used to pass arguments to functions.
- $v0 – $v1 (Value Registers): Used to store function return values.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Operands: Registers

Example:

```
add $t0, $s1, $s2
```

- $s1, $s2 → Source operands (saved registers)
- $t0 → Destination operand (temporary register)

Note: Other special-purpose registers such as **$sp** (Stack Pointer), **$fp** (Frame Pointer), and **$ra** (Return Address) will be discussed later during function calls and stack management.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Operands: Memory

- The 32 registers in MIPS are insufficient to store all program data.
- Main memory is used to store additional data and program instructions.
- Memory provides large storage capacity, but it is slower to access than registers.
- Frequently accessed variables are kept in registers, while less frequently used data is stored in memory.
- Data is transferred between memory and registers using load (lw) and store (sw) instructions.

Key Point:

Registers provide fast access for frequently used data, while memory provides large storage for the rest of the program.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory organization

Memories are organized as an array of data words.

The MIPS architecture uses **32-bit memory addresses and 32-bit data words.**

MIPS uses a byte-addressable memory.

That is, each byte in memory has a unique address.

However, for explanation purposes only, we first introduce a word-addressable memory, and afterward describe the MIPS byte-addressable memory.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory organization

A **32-bit address** means an address itself contains 32 bits (N).
Therefore, theoretically, MIPS can identify:

$$2^{32}$$

different addresses.
That is:

$$2^{32} = 4{,}294{,}967{,}296$$

different byte locations.
Since each location is **1 byte**, this corresponds to:

$$4 \text{ GB}$$

of byte-addressable memory.

Data

```
32-bit data word
= 4 bytes
```

Address

```
32-bit address
= identifies one byte location
```

So:

```
32-bit address → one BYTE
32-bit data word → four BYTES
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory organization

Let's take a real example
Suppose memory contains:

Address	Data
1000	AA
1001	BB
1002	CC
1003	DD

These four bytes together form one 32-bit word:
AA BB CC DD

Therefore:
Address 1000 ┐
Address 1001 │
Address 1002 ├──→ one 32-bit word
Address 1003 ┘

The next 32-bit word starts at address:
1004

This is why MIPS addresses often increase by 4
Suppose you have an array of 32-bit integers:
int A[0]
int A[1]
int A[2]
int A[3]……..int A[31]

Each integer is 4 bytes.

Therefore:
A[0] → address 0
A[1] → address 4
A[2] → address 8
A[3] → address 12

Notice:
0 → 4 → 8 → 12
not
0 → 1 → 2 → 3
because each integer occupies 4 bytes.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory organization: Word-Addressable Memory

Word-Addressable Memory: **A word-addressable memory** is a memory organization in which **each memory address refers to one complete word**. A word is the natural unit of data processed by the processor.

For a **32-bit processor**:

- **1 Word = 32 bits = 4 Bytes**
- Each address stores one complete 32-bit word.
- Memory addresses increase by **1 word**.

Characteristics

- One address points to one entire word.
- Efficient for processors that mainly operate on full words.
- Individual bytes cannot be directly addressed.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory organization: Word-Addressable Memory

Example:

Address	Stored data
0	Word 0
1	Word 1
2	Word 2

Key Point: In word-addressable memory, Address 2 means the **third word**, not the third byte.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Reading Word-Addressable Memory

Reading data from memory is called a **Load** operation.

MIPS uses the instruction:

lw destination, offset(base)

where:

- **lw** → Load Word
- **destination** → Register that receives the data
- **base** → Starting memory address
- **offset** → Displacement added to the base address

Address Calculation:

Effective Address=Base Register+Offset

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Reading Word-Addressable Memory

Example:

```
lw $s0, 5($t1)
```

If, $t1 = 20

Then, Address = 20 + 5 = 25

The word stored at address **25** is loaded into **$s0**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Reading Word-Addressable Memory: Example

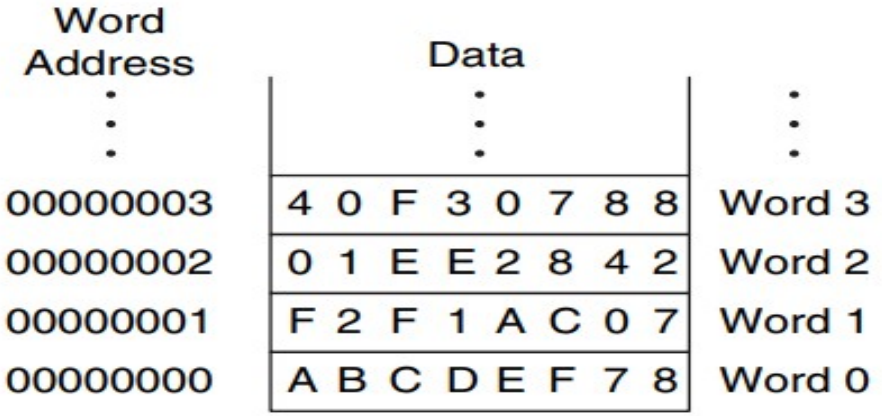
Example:

lw $s3, 1($0)

Step 1: Base Register, $0 = 0

Step 2: Offset: 1

Step 3: Effective Address, 0 + 1 = 1



Step 4: The processor reads the contents of **Memory Word 1**.

$s3 = 0xF2F1AC07

Observation:

Only one instruction is needed to transfer an entire **32-bit word** from memory to a register.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Writing Word-Addressable Memory

Writing data into memory is called a **Store** operation.

MIPS uses the instruction:

sw source, offset(base)

where:

- **sw** → Store Word
- **source** → Register containing data
- **base** → Base memory address
- **offset** → Address displacement

Address Calculation:

Effective Address=Base Register+Offset

The value from the register is copied into the calculated memory location.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Writing Word-Addressable Memory: Example

Example:

```
sw $t4, 0x7($0)   #write the value in $t4 to memory word 7
```

Step 1: Base Address, $0 = 0

Step 2: Offset, 0x7 = 7

Step 3: Effective Address, 0 + 7 = 7

Result:

The value stored in **$t4** is written into **Memory Word 7**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory Organization: Byte-Addressable Memory

Byte-Addressable Memory: A byte-addressable memory is a memory organization in which each memory address refers to **one byte (8 bits)** of data.

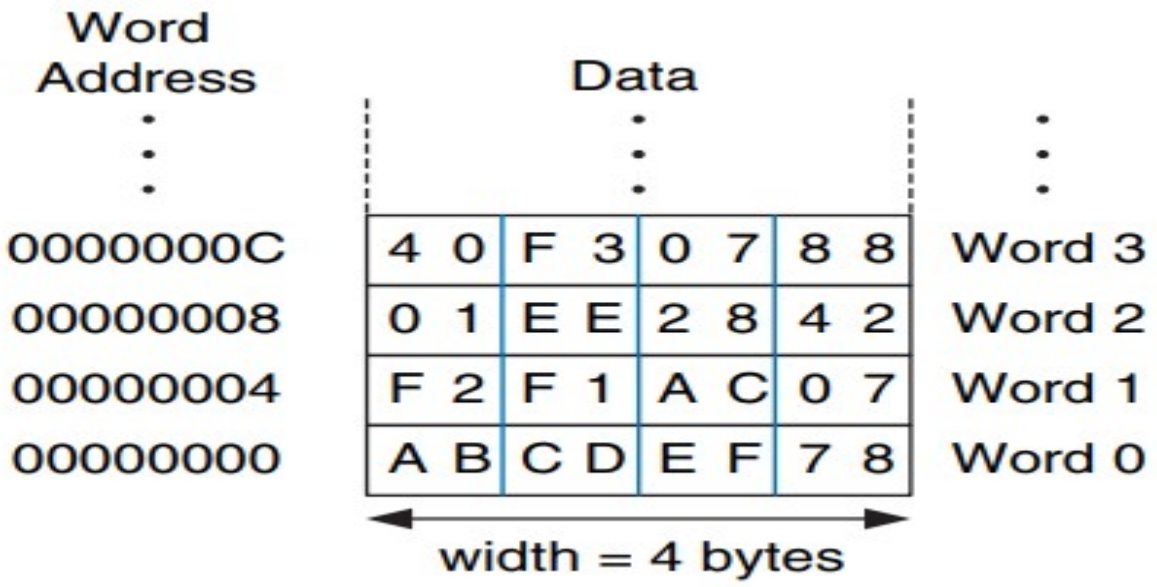
For a **32-bit processor**:
1 Byte = 8 bits
1 Word = 32 bits = 4 Bytes
Therefore, one 32-bit word occupies **four consecutive byte addresses**.

Characteristics:
- Each address identifies exactly one byte.
- Memory addresses increase by **1 byte** at a time.
- MIPS uses **byte-addressable memory**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Memory Organization: Byte-Addressable Memory



A 32-bit word consists of four 8-bit bytes. So each word address is a multiple of 4, as shown in Figure 6.2. Again, both the 32-bit word address and the data value are given in hexadecimal.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Byte-Addressable Memory: Example

Example:

Suppose one 32-bit word contains **0xF2F1AC07**.

In byte-addressable memory, this word occupies **four consecutive byte addresses**.

Byte Address	Meaning
0x04 – 0x07	**Four bytes belonging to one 32-bit word (0xF2F1AC07)**
0x08	Starting byte address of the next word
0x0C	Starting byte address of the following word

Key Point: The next 32-bit word begins at address **0x08**, because each word occupies **4 bytes**.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Reading Byte-Addressable Memory

Example:

```
lw $s3, 4($0)
```

Step 1: Base register, $0 = 0
Step 2: Offset = 4
Step 3: Effective address = 0 + 4 = 4
Step 4: The processor reads the **four bytes** stored at addresses **4, 5, 6, and 7**.
Result: The four bytes are combined to form one 32-bit word and loaded into $s3.
Observation: In MIPS, the offset in lw and sw is measured in bytes, not in words.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Word-Addressable vs Byte-Addressable Memory

Comparison:

- In **word-addressable memory**, one address refers to one complete word.
- In **byte-addressable memory**, one address refers to exactly one byte.
- In a 32-bit system, one word occupies **1 address** in word-addressable memory but **4 consecutive addresses** in byte-addressable memory.
- Word addresses increase by **1 word**, whereas byte addresses increase by **1 byte**.
- Byte-addressable memory is more flexible because individual bytes can also be accessed independently.

Key Point: MIPS uses **byte-addressable memory**, so consecutive 32-bit words begin at addresses such as **0, 4, 8, 12**.

"A MIPS word is 32 bits or 4 bytes, but MIPS assigns an address to every byte, not every word. Therefore, one 32-bit word occupies four consecutive memory addresses."

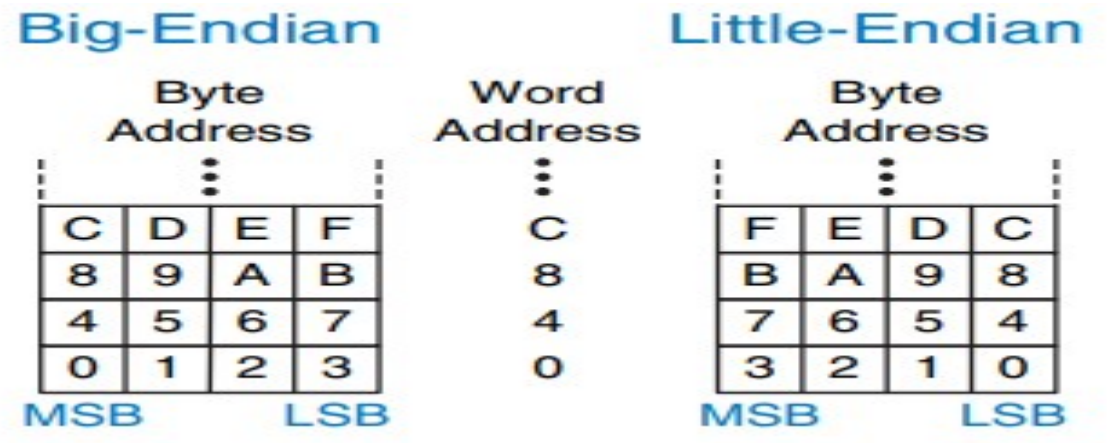
© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Big–Endian & Little–Endian Example

- How to number bytes within a word?
- **Little-endian**: byte numbers start at the little (least significant) end
- **Big-endian**: byte numbers start at the big (most significant) end
- Word address is the same for big- or little-endian

Big-Endian **Little-Endian**



© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Big–Endian & Little–Endian Example

• Jonathan Swift's Gulliver's Travels: the **Little-Endians** broke their eggs on the little end of the egg and the **Big-Endians** broke their eggs on the big end.

• It doesn't really matter which addressing type is used – except when the two systems need to share data!

Big-Endian

Byte Address

C	D	E	F
8	9	A	B
4	5	6	7
0	1	2	3

MSB LSB

Word Address

C
8
4
0

Little-Endian

Byte Address

F	E	D	C
B	A	9	8
7	6	5	4
3	2	1	0

MSB LSB

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Big–Endian & Little–Endian Example

- Suppose $t0 initially contains 0x23456789
- After following code runs on big–endian system, what value is in $s0?
- In a little–endian system?

```
sw $t0, 0($0)  // It stores the entire 32-bit value 0x23456789 starting at byte address 0.

lb $s0, 1($0)
```

- Big–endian: 0x00000045
- Little–endian: 0x00000067



Figure 6.4 Big-endian and little-endian data storage

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Design principle - 4

Good Design Demands Good Compromises

- Processor design involves balancing performance, cost, power consumption, and hardware complexity.
- Improving one aspect of a system often requires sacrificing another; therefore, practical designs rely on carefully chosen trade-offs.
- MIPS adopts a Reduced Instruction Set Computer (RISC) architecture by using a small set of simple instructions instead of many complex ones.
- This simplifies hardware implementation, improves execution speed, and reduces power consumption.
- An effective computer architecture achieves the best overall balance rather than optimizing a single parameter.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Design principle - 4

Example

RISC (MIPS):

- Simple instructions
- Faster execution
- Simpler hardware

CISC (x86):

- Complex instructions
- More hardware complexity
- Fewer instructions per program

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Operands: Constant (Immediate)

PES UNIVERSITY

Immediate Operands

- Constants are values **embedded directly within the instruction**, so they are immediately available during execution.
- MIPS uses a **16-bit signed (two's complement)** field to represent immediate values.
- Immediate operands eliminate the need to load small constants from memory, reducing the number of instructions.

Immediate Instructions

- **addi (Add Immediate):** Adds a constant directly to the value in a register.
- **andi, ori, xori:** Perform bitwise operations using immediate constants.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC &COMPUTER DESIGN

Operands: Constant (Immediate)

Why is there no subi instruction?

MIPS does **not** require a separate subi instruction because subtraction can be performed by adding a **negative immediate**.

Example:

```
addi $s1, $s0, -12
```

This instruction is equivalent to:

b = a - 12;

Key Point

Immediate instructions improve efficiency by encoding small constants directly in the instruction, avoiding an extra memory access or register for the constant.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Operands: Constant (Immediate)

C Code:

```
a = a + 4;
b = a – 12;
```

MIPS Assembly code:

```
# $s0 = a,  $s1 = b
addi $s0, $s0, 4
addi $s1, $s0, –12
```

Instruction Explanation:

- **`addi $s0, $s0, 4`** #Adds the immediate value **4** to `$s0`.
  - **Result:** `$s0 = $s0 + 4` (`a = a + 4`)
- **`addi $s1, $s0, -12`** #Adds **–12** to `$s0` and stores the result in `$s1`.
  - **Result:** `$s1 = $s0 – 12` (`b = a – 12`)

Note: `addi` uses a **16-bit signed immediate**, so subtraction is performed by adding a negative value.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Think About It - 1

Example 6.1 TRANSLATING HIGH-LEVEL CODE TO ASSEMBLY LANGUAGE
Translate the following high-level code into assembly language. Assume variables a –c are held in registers $s0–$s2 and f–j are in $s3–$s7.

a = b – c;

f = (g + h) − (i + j);

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Think About It - 1

Example 6.1 TRANSLATING HIGH-LEVEL CODE TO ASSEMBLY LANGUAGE
Translate the following high-level code into assembly language. Assume variables a –c are held in registers $s0–$s2 and f–j are in $s3–$s7.

a = b – c;

f = (g + h) – (i + j);

Solution: The program uses four assembly language instructions.

```
# MIPS assembly code
# $s0 = a, $s1 = b, $s2 = c, $s3 = f, $s4 = g, $s5 = h
# $s6 = i, $s7 = j
  sub $s0, $s1, $s2   # a = b - c
  add $t0, $s4, $s5   # $t0 = g + h
  add $t1, $s6, $s7   # $t1 = i + j
  sub $s3, $t0, $t1   # f = (g + h) - (i + j)
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Think About It - 2

The code shows how to read and write words in the MIPS byte-addressable memory. The word address is four times the word number. The MIPS assembly code reads words 0, 2, and 3 and writes words 1, 8, and 100. The offset can be written in decimal or hexadecimal.

Code Example 6.8 ACCESSING BYTE-ADDRESSABLE MEMORY

MIPS Assembly Code

```
lw $s0, 0($0)        # read data word 0 (0xABCDEF78) into $s0
lw $s1, 8($0)        # read data word 2 (0x01EE2842) into $s1
lw $s2, 0xC($0)      # read data word 3 (0x40F30788) into $s2
sw $s3, 4($0)        # write $s3 to data word 1
sw $s4, 0x20($0)     # write $s4 to data word 8
sw $s5, 400($0)      # write $s5 to data word 100
```

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Think About It - 3

Using the instructions learnt so far (add and sub), can you write a program (sequence of instructions) that would left shift a number (stored in some register) by three bit positions? How many registers do you need?

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Think About It - 3

Using the instructions learnt so far (add and sub), can you write a program (sequence of instructions) that would left shift a number (stored in some register) by three bit positions? How many registers do you need?

Solution:

Left shifting a number by 1 bit is equivalent to multiplying it by 2.

So, left shifting by 3 bits = multiplying by $2^3 = 8$.

We can do this using only add and sub.

Example

Let the number in $t0 be $N = 13$ $(0000\ 1101)_2$

Step	Register	Binary Value	Decimal Value
1	$t1	0001 1010	26
2	$t2	0011 0100	52
3	$t3	0110 1000	104

Sequence of Instructions (MIPS)

Step	Instruction	Comment
1	add $t1, $t0, $t0	$t1 = $t0 + $t0 → 2 × N
2	add $t2, $t1, $t1	$t2 = $t1 + $t1 → 4 × N
3	add $t3, $t2, $t2	$t3 = $t2 + $t2 → 8 × N (left shift by 3)

Registers Used: $t0, $t1, $t2, $t3

Total Registers Needed: 4

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Think about it - 4

- By writing just an assembly language program, can you determine if the processor it is running on is Big–Endian or Little–Endian?

- What if the processor has no load byte and store byte instructions?

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

Think about it - 4

Can an Assembly Program Determine Endianness?

YES.

If the processor provides **byte-level memory access instructions** (e.g., `lb` / `sb`), **an assembly program can determine** whether it is **Big–Endian** or **Little–Endian**.

Method:

1. Store the 32-bit value 0x12345678 at memory address 1000.
2. Load the first byte from address 1000.

- If it reads **0x12** → **Big–Endian**
- If it reads **0x78** → **Little–Endian**

Example: Store 0x12345678 starting at address 1000

Big–Endian

Address	Byte Stored
1000	12
1001	34
1002	56
1003	78

Little–Endian

Address	Byte Stored
1000	78
1001	56
1002	34
1003	12

Program executes: `lb $t0, 0($t1)` # load byte from address 1000

- If $t0 = **0x12** → **Big–Endian**
- If $t0 = **0x78** → **Little–Endian**

NO.

If the processor has no instructions for accessing individual bytes (only word load/store, e.g., `lw` / `sw`), an assembly program generally cannot determine endianness.

Why?

Endianness only affects the order of bytes within a multi-byte word in memory.

If we can only store and retrieve complete words, the internal byte ordering is not observable.

Example (both Big–Endian and Little–Endian):

Program

```
li   $t0, 0x12345678
sw   $t0, 0($t1)    # store word
lw   $t2, 0($t1)    # load word
```

Result: $t2 = 0x12345678

Big–Endian Memory

Addr	Byte
A	12
A+1	34
A+2	56
A+3	78

Little–Endian Memory

Addr	Byte
A	78
A+1	56
A+2	34
A+3	12

Conclusion:

- With byte-level access (`lb` / `sb`): **Yes,** endianness can be determined.
- Without byte–level access: **No,** the difference is invisible to the program.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

- **Q1.An operand stored in memory is accessed using which instructions in MIPS?**

  A) add and sub

  B) lw and sw

  C) beq and j

  D) and and or

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

- **Q2. Memory is primarily used to store:**

  A) Frequently used temporary values only

  B) Large amounts of data and programs

  C) CPU clock signals

  D) ALU outputs only

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

- **Q3. In byte-addressable memory, each memory address represents:**

  A) One bit

  B) One byte

  C) One word

  D) Eight bytes

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

- **Q4. In word-addressable memory, each memory address represents:**

  A) One bit

  B) One byte

  C) One word

  D) Eight bytes

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

- **Q5. An operand that is directly specified within an instruction is called a:**
  - A) Memory operand
  - B) Register operand
  - C) Constant (Immediate) operand
  - D) Address operand

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

- **Q6. Which statement correctly distinguishes word-addressable and byte-addressable memory?**

  A) Both assign one address per word.

  B) Both assign one address per byte.

  C) Word-addressable memory assigns one address per word, while byte-addressable memory assigns one address per byte.

  D) Byte-addressable memory cannot store words.

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.

DIGITAL LOGIC & COMPUTER DESIGN

MCQ

Answer Key

Q1: B **Q2: B** **Q3: B** **Q4: C** **Q5: C** **Q6: C**

© Elsevier, Digital Design and Computer Architecture, 2nd Ed., 2012.



THANK YOU

Team DLCD

Department of CSE and CSE(AI & ML)